Workshop 9: Processing data with Pandas

FIE463: Numerical Methods in Macroeconomics and Finance using Python

Richard Foltyn
NHH Norwegian School of Economics

March 19, 2026

See GitHub repository for notebooks and data:

https://github.com/richardfoltyn/FIE463-V26

Exercise 1: Data cleaning

Before doing actual data analysis, we usually first need to clean the data. This might involve steps such as dealing with missing values and encoding categorical variables as integers.

Load the Titanic dataset in `titanic.csv` and perform the following tasks:

1. Report the number of observations with missing `Age`, for example, using `isna()`.
2. Compute the average age in the dataset. Use the following approaches and compare your results:
    1. Use the `mean()` method.
    2. Convert the `Age` column to a NumPy array using `to_numpy()`. Experiment with NumPy's `np.mean()` and `np.nanmean()` to see if you obtain the same results.
3. Replace all missing ages with the mean age you computed above, rounded to the nearest integer. Convert this updated `Age` column to integer type using `astype()`.

    Note that in "real" applications, replacing missing values with sample means is usually not a good idea.
4. Generate a new column `Female` which takes on the value one if `Sex` is equal to `"female"` and zero otherwise. This is called an *indicator* or *dummy* variable, and is preferable to storing such categorical data as strings. Delete the original column `Sex`.
5. Save your cleaned dataset as `titanic-clean.csv` using `to_csv()` with `,` as the field separator. Tell `to_csv()` to *not* write the `DataFrame` index to the CSV file as it's not needed in this example.

Solution.

Loading the data

[1]:
```
# Path to data directory
DATA_PATH = '../../data'

# Alternatively, load data directly from GitHub
# DATA_PATH = 'https://raw.githubusercontent.com/richardfoltyn/FIE463-V26/main/data'
```

[2]:
```
import pandas as pd

# Path to Titanic CSV file
```

```
fn = f'{DATA_PATH}/titanic.csv'

df = pd.read_csv(fn)
```

Part 1 — Number of missing values

The number of non-missing values can be displayed using the `info()` method. Alternatively, we can count the number of missing values directly by summing the return values of `isna()`.

[3]:
```
# Display missing counts for each column
df.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 10 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   Ticket       891 non-null    object 
 7   Fare         891 non-null    float64
 8   Cabin        204 non-null    object 
 9   Embarked     889 non-null    object 
dtypes: float64(2), int64(3), object(5)
memory usage: 69.7+ KB
```

[4]:
```
# Alternative way to get the number of missing values:
df['Age'].isna().sum()
```

[4]:
```
np.int64(177)
```

Part 2 — Compute mean age

We compute the mean age using the three different methods. As you can see, `np.mean()` cannot deal with missing values and returns `NaN` (*"not a number"*).

[5]:
```
import numpy as np

# Compute mean age using the DataFrame.mean() method
mean_age = df['Age'].mean()

# Convert Age column to NumPy array
age_array = df['Age'].to_numpy()

# Compute mean using np.mean()
mean_age_np = np.mean(age_array)

# Compute mean using np.nanmean()
mean_age_np_nan = np.nanmean(age_array)

print(f'Mean age using pandas:       {mean_age:.3f}')
print(f'Mean age using np.mean():    {mean_age_np:.3f}')
print(f'Mean age using np.nanmean(): {mean_age_np_nan:.3f}')
```

```
Mean age using pandas:       29.699
Mean age using np.mean():    nan
Mean age using np.nanmean(): 29.699
```

Part 3 — Replace missing values

There are several ways to replace missing values. First, we can "manually" identify these using boolean indexing and assign a new value to such observations.

[6]:
```
# Round average age
mean_age = np.round(mean_age)

# boolean arrays to select missing observations
is_missing = df['Age'].isna()

# Update missing observations with rounded mean age
df.loc[is_missing, 'Age'] = mean_age
```

There is also the convenience routine `fillna()` which automates this step. To illustrate, we need to reload the original data as we have just replaced all missing values.

[7]:
```
# Re-load data to get the original missing values
df = pd.read_csv(fn)

df['Age'] = df['Age'].fillna(value=mean_age)
```

Since age is usually recorded as an integer, there is no reason to store it as a `float` once we have dealt with the missing values.

[8]:
```
df['Age'] = df['Age'].astype(int)
```

Part 4 — Generate Female indicator

An indicator variable can be obtained as a result of a logical operation (`==`, `!=`, etc.). This value contains `True` or `False` values, which we can convert to 1 or 0 by changing the data type to integer.

[9]:
```
# Generate boolean array (True/False) whether passenger is female
is_female = (df['Sex'] == 'female')

# Add Female dummy variable, converted to integer
df['Female'] = is_female.astype(int)

# Delete original Sex column, no longer needed
del df['Sex']

# Alternatively, you can use
# df = df.drop(columns=['Sex'])
```

Part 5 — Save cleaned file

We can use `info()` again to confirm that `Age` has no missing values and all columns are of the desired data type:

[10]:
```
df.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 10 columns):
 #   Column       Non-Null Count  Dtype
```

```
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Age          891 non-null    int64
 5   Ticket       891 non-null    object
 6   Fare         891 non-null    float64
 7   Cabin        204 non-null    object
 8   Embarked     889 non-null    object
 9   Female       891 non-null    int64
dtypes: float64(1), int64(5), object(4)
memory usage: 69.7+ KB
```

[11]:
```python
# Save cleaned file
fn_clean = 'titanic-clean.csv'

df.to_csv(fn_clean, sep=',', index=False)
```

Exercise 2: Daily returns of US stock market indices

In this exercise, we examine how the three major US stock market indices performed last year using data from Yahoo! Finance.

1. Use the `yfinance` library and its `download()` function to obtain the time series of daily observations for the S&P 500, the Dow Jones Industrial Average (DJIA) and the NASDAQ Composite indices. Restrict the sample to the period from 2025-01-01 to 2025-12-31 and keep only the closing price stored in column `Close`.

   Hint: The corresponding ticker symbols are `^GSPC`, `^DJI`, `^IXIC`, respectively.

2. Rename the DataFrame columns to `'SP500'`, `'Dow Jones'` and `'NASDAQ'` using the `rename()` method.

   Hint: `rename(columns=dict)` expects a dictionary as an argument which maps existing to new column names.

3. Plot the three time series (one for each index) in a single graph. Label all axes and make sure your graph contains a legend.

   Hint: You can directly use the `DataFrame.plot()` method implemented in `pandas`.

4. The graph you created in the previous sub-question is not well-suited to illustrate how each index developed in 2025 since the indices are reported on vastly different scales (the S&P 500 appears to be an almost flat line).

   To get a better idea about how each index fared in 2025 relative to its value at the beginning of the year, normalize each index by its value on the first trading day in 2025 (which was 2025-01-02). Plot the resulting normalized indices.

5. For each index, compute the daily returns, i.e., the relative change vs. the previous closing price in percent. Create a plot of the daily returns for all indices.

   Hint: Use `pct_change()` to compute the change relative to the previous observation.

Solution.

Part 1 — Download data

[12]:
```
import yfinance as yf

# List of ticker symbols
tickers = ['^GSPC', '^DJI', '^IXIC']

# Period
start = '2025-01-01'
end = '2025-12-31'

data = yf.download(tickers, start=start, end=end)
```

```
[                       0%                       ]
[**********************67%*******                ]  2 of 3 completed
[**********************100%**********************]  3 of 3 completed
```

The resulting DataFrame has a hierarchical column index, with the first level being the variable names (`Adj Close`, `Close`, etc.) and the second level consisting of the ticker symbols. This can be seen if we inspect the columns of the DataFrame using the `info()` method:

[13]:
```
data.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 249 entries, 2025-01-02 to 2025-12-30
Data columns (total 15 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   (Close, ^DJI)    249 non-null    float64
 1   (Close, ^GSPC)   249 non-null    float64
 2   (Close, ^IXIC)   249 non-null    float64
 3   (High, ^DJI)     249 non-null    float64
 4   (High, ^GSPC)    249 non-null    float64
 5   (High, ^IXIC)    249 non-null    float64
 6   (Low, ^DJI)      249 non-null    float64
 7   (Low, ^GSPC)     249 non-null    float64
 8   (Low, ^IXIC)     249 non-null    float64
 9   (Open, ^DJI)     249 non-null    float64
 10  (Open, ^GSPC)    249 non-null    float64
 11  (Open, ^IXIC)    249 non-null    float64
 12  (Volume, ^DJI)   249 non-null    int64
 13  (Volume, ^GSPC)  249 non-null    int64
 14  (Volume, ^IXIC)  249 non-null    int64
dtypes: float64(12), int64(3)
memory usage: 31.1 KB
```

The columns in the hierarchical `MultiIndex` are difficult to work with, so we only keep the `Close` column and discard the remaining data:

[14]:
```
# Keep only Close column
data = data['Close']
```

Part 2 — Rename columns

We get rid of the inconvenient column names and replace them with something more readable:

[15]:
```
# Rename to get nicer column names
names = {
    '^GSPC': 'SP500',
```

```
    '^DJI': 'Dow Jones',
    '^IXIC': 'NASDAQ'
}

data = data.rename(columns=names)
```

We can get a peek at the first few rows using the `head()` method.

[16]:
```
data.head(3)
```

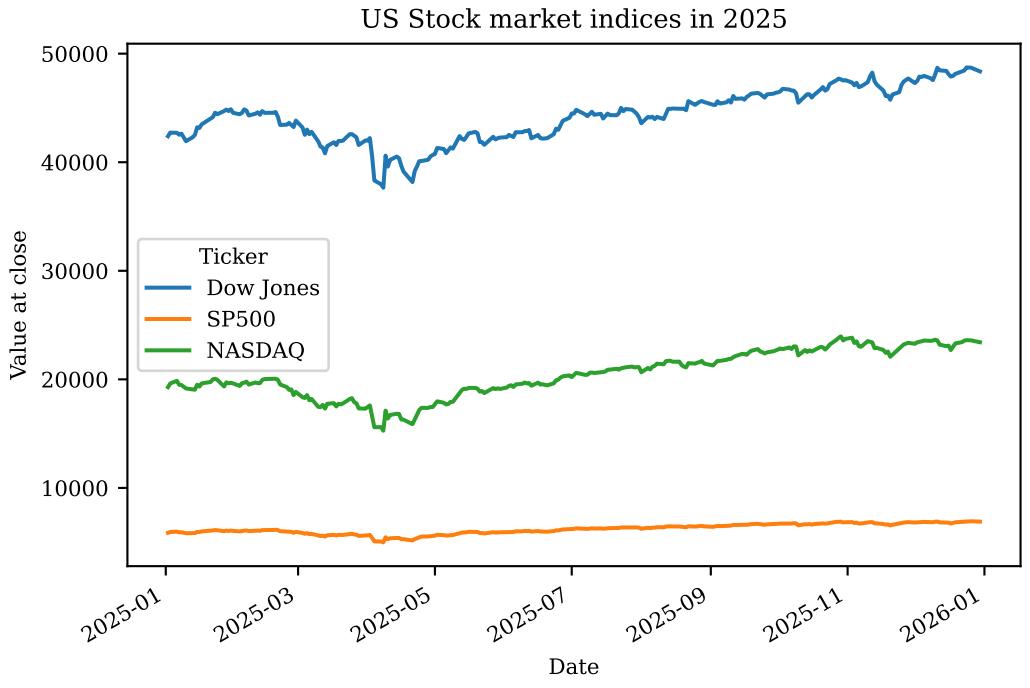
[16]:
```
Ticker         Dow Jones        SP500        NASDAQ
Date
2025-01-02  42392.269531  5868.549805  19280.789062
2025-01-03  42732.128906  5942.470215  19621.679688
2025-01-06  42706.558594  5975.379883  19864.980469
```

Part 3 — Plot indices

We can create the plot directly with pandas's plotting functions:

[17]:
```
# Plot all three indices, setting a label for the y-axis.
data.plot(
    ylabel='Value at close', title='US Stock market indices in 2025', figsize=(6, 4)
)
```

[17]:
```
<Axes: title={'center': 'US Stock market indices in 2025'}, xlabel='Date',
ylabel='Value at close'>
```

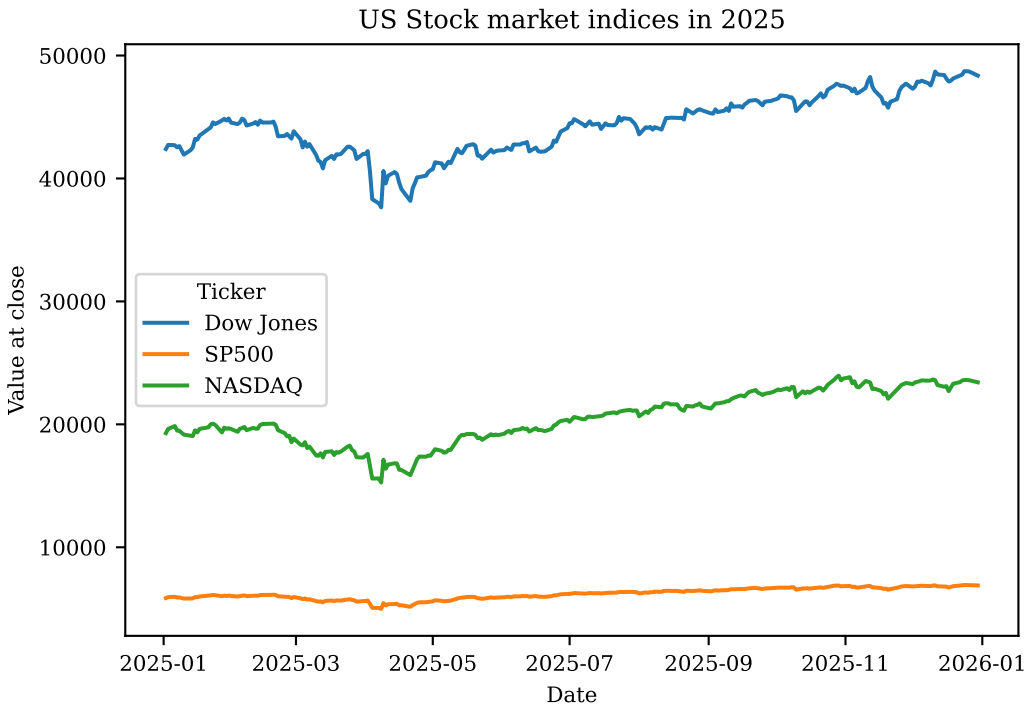


Alternatively, we can create the plot ourselves using traditional Matplotlib functions:

[18]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.plot(data)
plt.legend(data.columns, title='Ticker')
plt.xlabel('Date')
```

```
plt.ylabel('Value at close')
plt.title('US Stock market indices in 2025')
```

[18]:
```
Text(0.5, 1.0, 'US Stock market indices in 2025')
```



Part 4 — Normalized indices

To normalize each column by its first value, divide the `DataFrame` by its first row which can be selected using `.iloc[]`:

[19]:
```
close_norm = data / data.iloc[0]
```

You can use `head()` to verify that the first normalized element of each column is now 1.

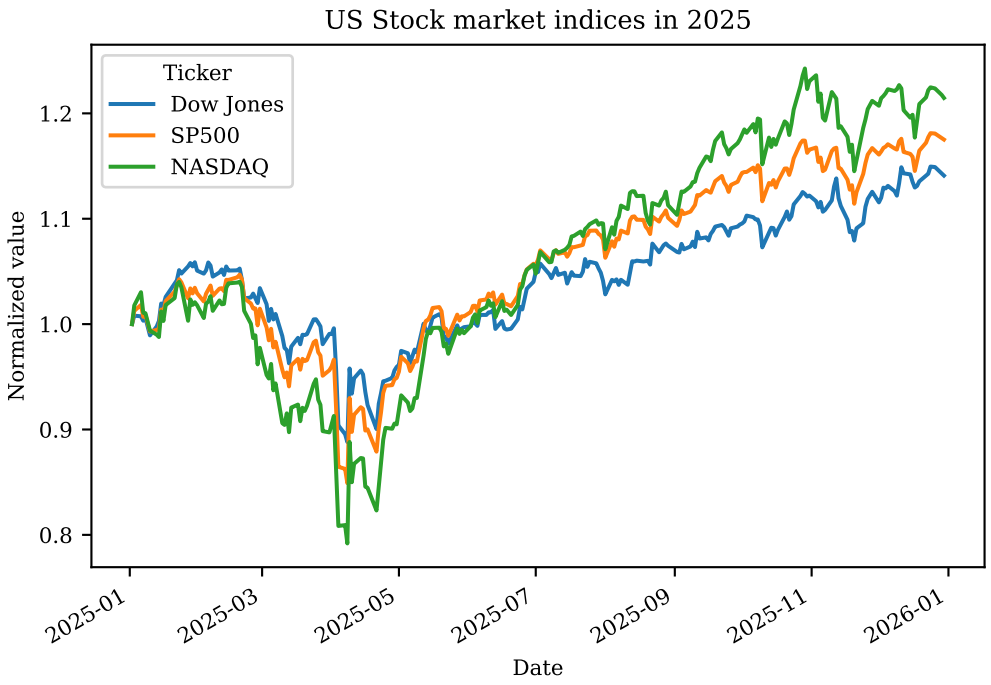
[20]:
```
close_norm.head(3)
```

[20]:
```
Ticker      Dow Jones     SP500    NASDAQ
Date
2025-01-02   1.000000  1.000000  1.000000
2025-01-03   1.008017  1.012596  1.017680
2025-01-06   1.007414  1.018204  1.030299
```

Finally, we plot the normalized indices just like in the previous sub-question. It is now much easier to see that these indices moved very similarly during this year.

[21]:
```
# Create a plot using pandas's plotting functions
close_norm.plot(
    ylabel='Normalized value', title='US Stock market indices in 2025', figsize=(6, 4)
)
```

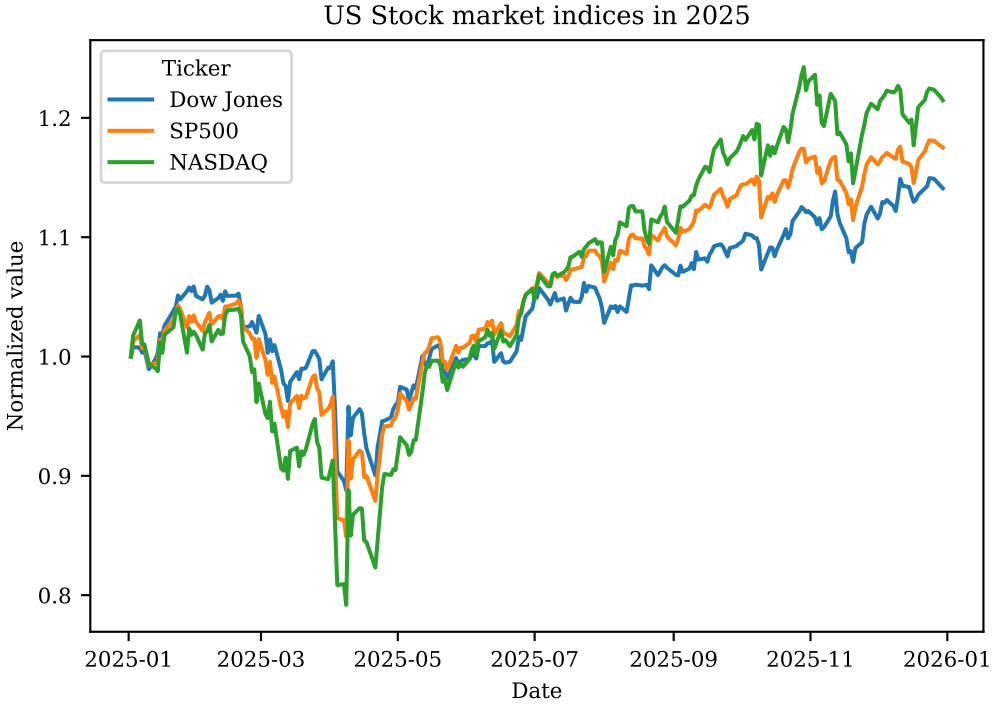
[21]:
```
<Axes: title={'center': 'US Stock market indices in 2025'}, xlabel='Date',
ylabel='Normalized value'>
```



Alternatively, we can again create the plot using Matplotlib functions:

[22]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.plot(close_norm)
plt.legend(close_norm.columns, title='Ticker')
plt.xlabel('Date')
plt.ylabel('Normalized value')
plt.title('US Stock market indices in 2025')
```

[22]:
```
Text(0.5, 1.0, 'US Stock market indices in 2025')
```



Part 5 — Daily returns

We use the `pct_change()` method to compute the relative difference between two consecutive closing prices.

[23]:
```
# Relative difference from previous closing price in percent
returns = data.pct_change() * 100.0
```

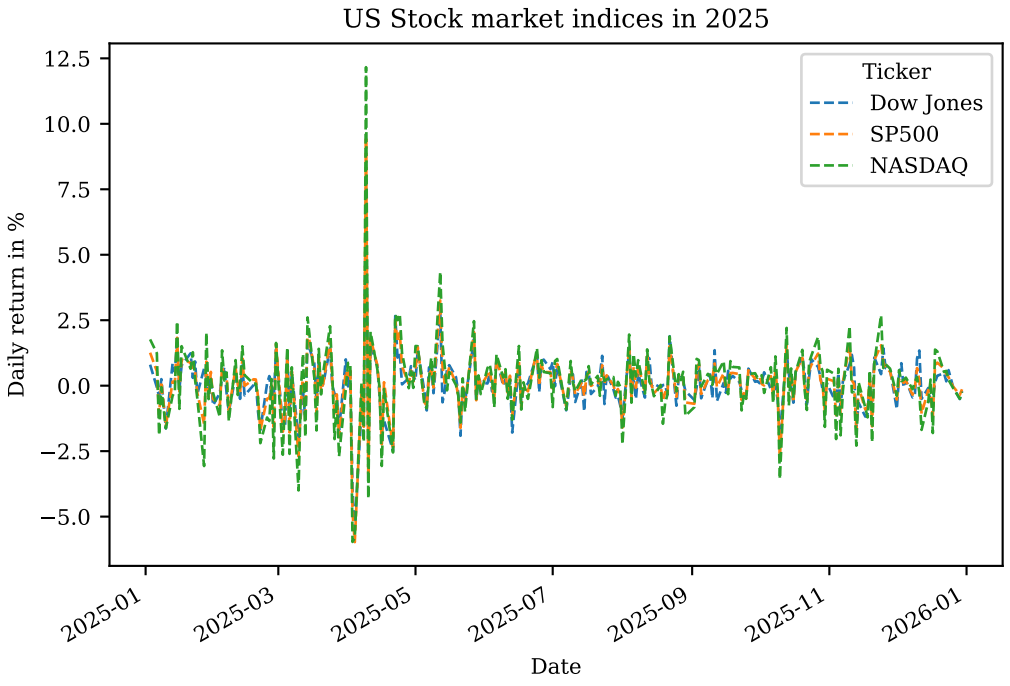
Because we cannot compute a difference for the very first observation, this value is set to `NaN`.

[24]:
```
returns.head(3)
```

[24]:
```
Ticker      Dow Jones     SP500    NASDAQ
Date
2025-01-02        NaN       NaN       NaN
2025-01-03   0.801701  1.259603  1.768033
2025-01-06  -0.059839  0.553805  1.239959
```

[25]:
```
# use dashed lines since daily returns are overlapping
returns.plot(
    ylabel='Daily return in %',
    lw=1.0,
    ls='--',
    figsize=(6, 4),
    title='US Stock market indices in 2025',
)
```

[25]:
```
<Axes: title={'center': 'US Stock market indices in 2025'}, xlabel='Date',
ylabel='Daily return in %'>
```

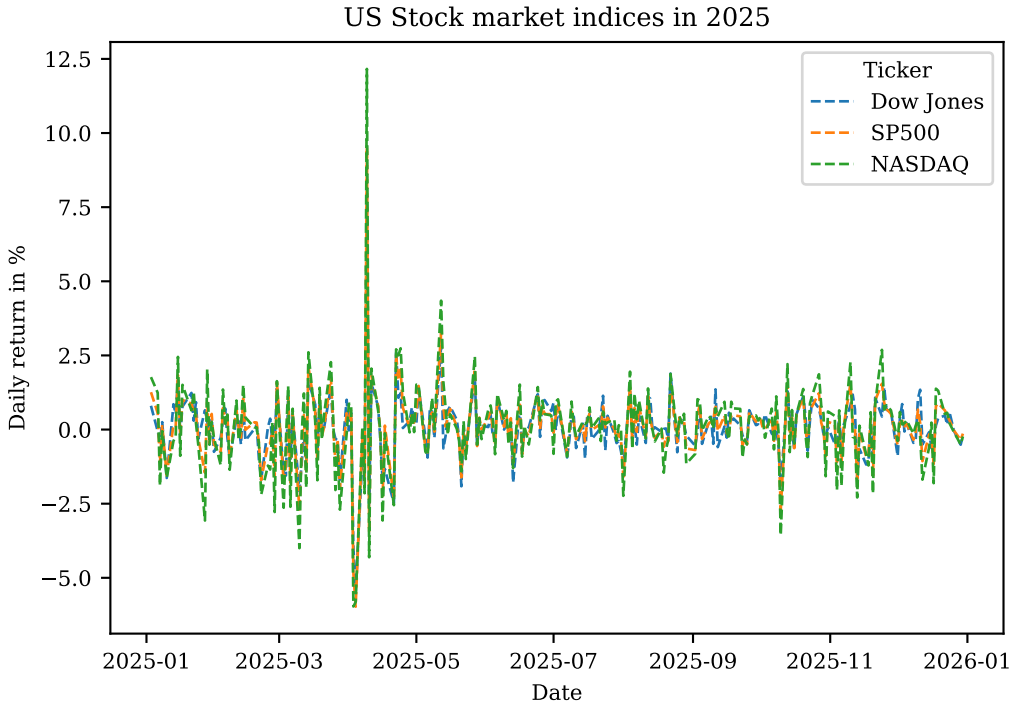


The corresponding Matplotlib code to create this graph is as follows:

[26]:
```
plt.figure(figsize=(6, 4))
plt.plot(returns, ls='--', lw=1)
plt.legend(returns.columns, title='Ticker')
plt.xlabel('Date')
plt.ylabel('Daily return in %')
```

```
plt.title('US Stock market indices in 2025')
```

[26]: `Text(0.5, 1.0, 'US Stock market indices in 2025')`



Exercise 3: Weekly returns of the Magnificent Seven

In this exercise, you are asked to analyze the weekly stock market returns of the so-called Magnificent Seven, which are some of the most successful tech companies of the last decades: Apple (AAPL), Amazon (AMZN), Google (GOOG), Meta (META), Microsoft (MSFT), Nvidia (NVDA), and Tesla (TSLA).

1. Load the CSV data from `../../data/stockmarket/magnificent7.csv`. Inspect the first few rows to familiarize yourself with the columns present in the `DataFrame`.

   - Keep only the columns `Date`, `Ticker`, `Open`, and `Close`.
   - Convert the `Date` column to datetime format using `pd.to_datetime()`. Alternatively, you can specify the `parse_dates` argument when loading the data with `read_csv()`.

2. You want to compute weekly returns for each of the 7 stocks. To this end, you need to reshape the `DataFrame` so that `Date` is the index and the remaining dimensions are in (hierarchical) columns.

   One way to achieve this is to use the `pivot()` function (also see the graphical illustration in the user guide). Call this function with the arguments `index='Date'` and `columns='Ticker'` and inspect the result.

   This should generate a hierarchical column index with `Open` and `Close` at the top level.

   Drop all rows with any missing values which arise because these stocks have been listed at different points in time.

3. Your data is now in a format that can be resampled to weekly frequency. Use `resample()` to convert the data to weekly observations.

   Compute the weekly returns as the relative difference between the *first* `Open` quote and the *last* `Close` quote for each ticker in each week:

$$\text{Weekly returns} = \frac{\text{Close price on last day} - \text{Open price on first day}}{\text{Open price on first day}} \times 100$$

   Hint: For example, to select the first `Open` value in each week, you should use `resample('W')['Open'].first()`.

4. Create a 3-by-3 figure and plot the weekly returns you computed for each ticker as a histogram, using 25 bins (i.e., `bins=25` should be passed to the `hist()` function).

   Since you have only 7 tickers but 9 subplots in the figure, the last two remaining subplots should remain empty.

   Hint: You can either use `DataFrame.hist()` to plot the histogram, or Matplotlib's `hist()` function. In either case, you should add `density=True` such that the histogram is appropriately rescaled and comparable to the normal density.

5. **[Advanced]** Compare the histograms you created to the normal (Gaussian) probability density function (PDF) to get an idea of how much weekly returns differ from a normal distribution.

   First, compute mean and standard deviation for each ticker and tabulate these.

   Then add a line showing the normal PDF to each of the return histograms you created previously, using the mean and standard deviation for each ticker.

   Hint: Use the `pdf()` method of the `scipy.stats.norm` class to compute the normal density.

6. Finally, you are interested in how the weekly returns are correlated across the 7 stocks.

   Create a figure with 7-by-7 subplots showing the pairwise correlations for each combination of stocks.

   You can do this either with the `scatter_matrix()` function contained in `pandas.plotting`, or build the figure using Matplotlib functions.

   [Advanced] Additionally, use the DataFrame.corr() method to compute the pairwise correlation matrix. Extract these values and add them as text to each of the 7-by-7 subplots (e.g., the correlation between returns on AAPL and AMZN is about 0.43, so this text should be added to the subplot showing the scatter plot of AAPL vs. AMZN).

Solution.

Part 1 — Load data

[27]:
```
# Path to data directory
DATA_PATH = '../../data/'
```

[28]:
```
import pandas as pd

# Path to CSV file with stock market data
file = f'{DATA_PATH}/stockmarket/magnificent7.csv'

# Load data from CSV file
df = pd.read_csv(file, parse_dates=['Date'])

# Keep only selected columns
df = df[['Date', 'Ticker', 'Open', 'Close']]
```

```
# Display first 3 rows of the DataFrame
df.head(3)
```

[28]:
```
        Date Ticker    Open   Close
0 1980-12-12   AAPL  0.0984  0.0984
1 1980-12-15   AAPL  0.0937  0.0933
2 1980-12-16   AAPL  0.0868  0.0864
```

Part 2 — Pivot data

We first move the ticker dimension to the column axis so that the unique date can be used as the row index:

[29]:
```
# Pivot Ticker to columns, set Date as index
df = df.pivot(index='Date', columns='Ticker')
```

This generates rows with missing values because the panel of stock quotes is not balanced. We discard all dates which are missing observations for any of the stocks.

[30]:
```
df = df.dropna()
```

The layout of the resulting `DataFrame` is now as follows:

[31]:
```
df.head(3)
```

[31]:
```
               Open                                                  \
Ticker         AAPL     AMZN    GOOGL     META     MSFT    NVDA    TSLA
Date
2012-05-18  16.0140  10.9705  15.5258  41.7583  23.6383  0.2906  1.8913
2012-05-21  16.0302  10.7015  14.9151  36.2766  23.0908  0.2773  1.8387
2012-05-22  17.0814  10.9155  15.2362  32.3838  23.5589  0.2815  2.0067

              Close
Ticker         AAPL     AMZN    GOOGL     META     MSFT    NVDA    TSLA
Date
2012-05-18  15.9067  10.6925  14.9124  37.9648  23.2257  0.2769  1.8373
2012-05-21  16.8334  10.9055  15.2529  33.7939  23.6066  0.2817  1.9180
2012-05-22  16.7041  10.7665  14.9223  30.7850  23.6145  0.2783  2.0533
```

Part 3 — Resample to weekly frequency

We create a `Resampler` object by calling `resample()`. This object is similar to the one returned by `groupby()`. In particular, we can select individual columns and apply aggregation operations:

[32]:
```
resampler = df.resample('W')
```

For example, the following code selects the first `Open` observation in each week and prints the first 3 observations:

[33]:
```
resampler['Open'].first().head(3)
```

[33]:
```
Ticker         AAPL     AMZN    GOOGL     META     MSFT    NVDA    TSLA
Date
2012-05-20  16.0140  10.9705  15.5258  41.7583  23.6383  0.2906  1.8913
2012-05-27  16.0302  10.7015  14.9151  36.2766  23.0908  0.2773  1.8387
2012-06-03  17.1219  10.7150  14.7983  31.2616  23.3130  0.2888  2.0007
```

Similarly, the code below selects the last `Close` observation:

[34]:
```
resampler['Close'].last().head(3)
```

[34]:
```
Ticker         AAPL     AMZN    GOOGL     META     MSFT    NVDA    TSLA
Date
2012-05-20  15.9067  10.6925  14.9124  37.9648  23.2257  0.2769  1.8373
2012-05-27  16.8637  10.6445  14.6920  31.6886  23.0590  0.2842  1.9873
2012-06-03  16.8247  10.4110  14.1816  27.5277  22.5750  0.2746  1.8767
```

Note that the date index is the same for both DataFrames even though these observations clearly belong to different week days (usually Monday vs. Friday). The reason is that the Date index is determined by the resampling specification `'W'`, *not* by the aggregating operation applied to the data.

We can now compute the weekly returns as the relative difference between the last `Close` quote and the first `Open` quote within any given week:

[35]:
```
# Weekly returns in percent
returns = ((resampler['Close'].last() /  resampler['Open'].first()) - 1) * 100

# Verify that results look as expected
returns.head(3)
```

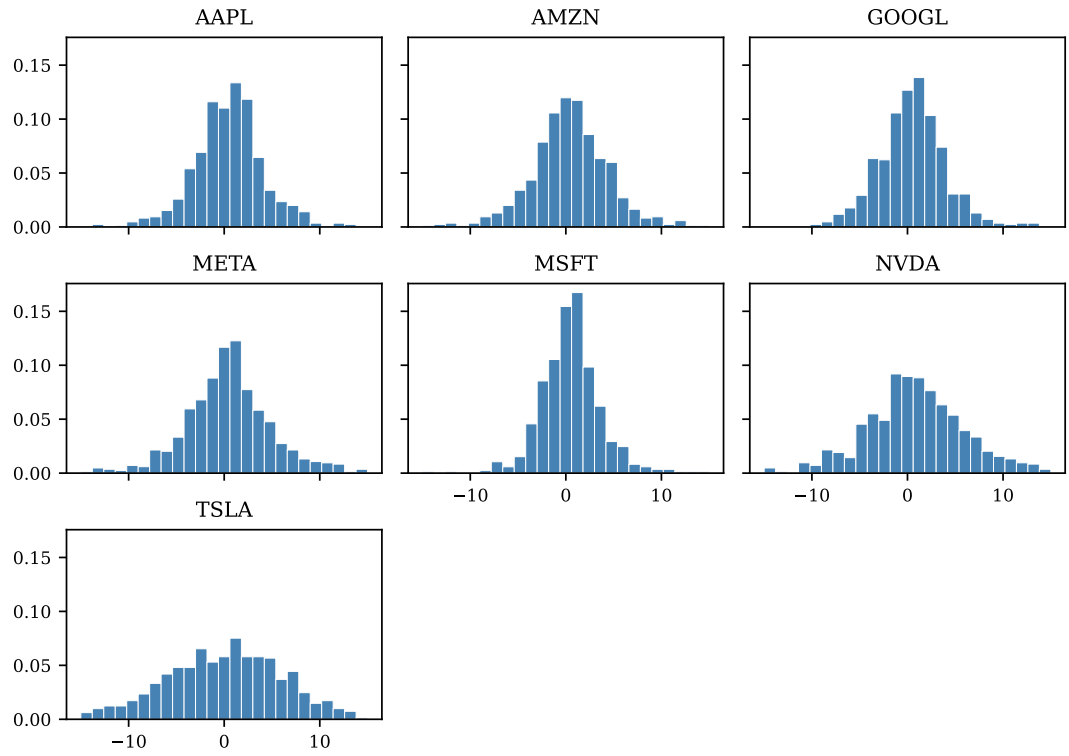
[35]:
```
Ticker          AAPL      AMZN      GOOGL       META      MSFT      NVDA  \
Date
2012-05-20 -0.670039 -2.534069 -3.950843  -9.084422 -1.745472 -4.714384
2012-05-27  5.199561 -0.532636 -1.495800 -12.647271 -0.137717  2.488280
2012-06-03 -1.735789 -2.837144 -4.167371 -11.944046 -3.165616 -4.916898

Ticker          TSLA
Date
2012-05-20 -2.855179
2012-05-27  8.081797
2012-06-03 -6.197831
```

Part 4 — Plot histograms of weekly returns

The histograms can be generated either using `DataFrame.hist()` or by building the figure from scratch with Matplotlib functions.

We demonstrate the first approach in this part and the second one below when we add the normal density to each plot.

[36]:
```
xmin, xmax = -15, 15

# Use DataFrame.hist() to create histograms
axes = returns.hist(
    bins=25,                # Number of bins in histogram
    figsize=(7, 5),         # Set size of the figure
    grid=False,             # Turn off grid
    layout=(3, 3),          # 3x3 grid of subplots
    sharex=True,            # Share x-axis range
    sharey=True,            # Share y-axis range
    density=True,           # Normalize histograms to densities
    range=(xmin, xmax),     # Set range for x-axis
    color='steelblue',      # Color for bars
    edgecolor='white',      # Color for bar edges
    lw=0.4,                 # Width of bar edges
)

# Use array of Axes to get figure and adjust layout
axes[0, 0].figure.tight_layout()
```



Part 5 — Add normal density

We first compute the mean and standard deviation of the returns for each ticker. We can do this with a single `agg()` call, passing a list of functions to be applied to the data.

[37]:
```
# Compute mean and standard deviation of returns
moments = returns.agg(['mean', 'std'])

# Display the moments
moments
```

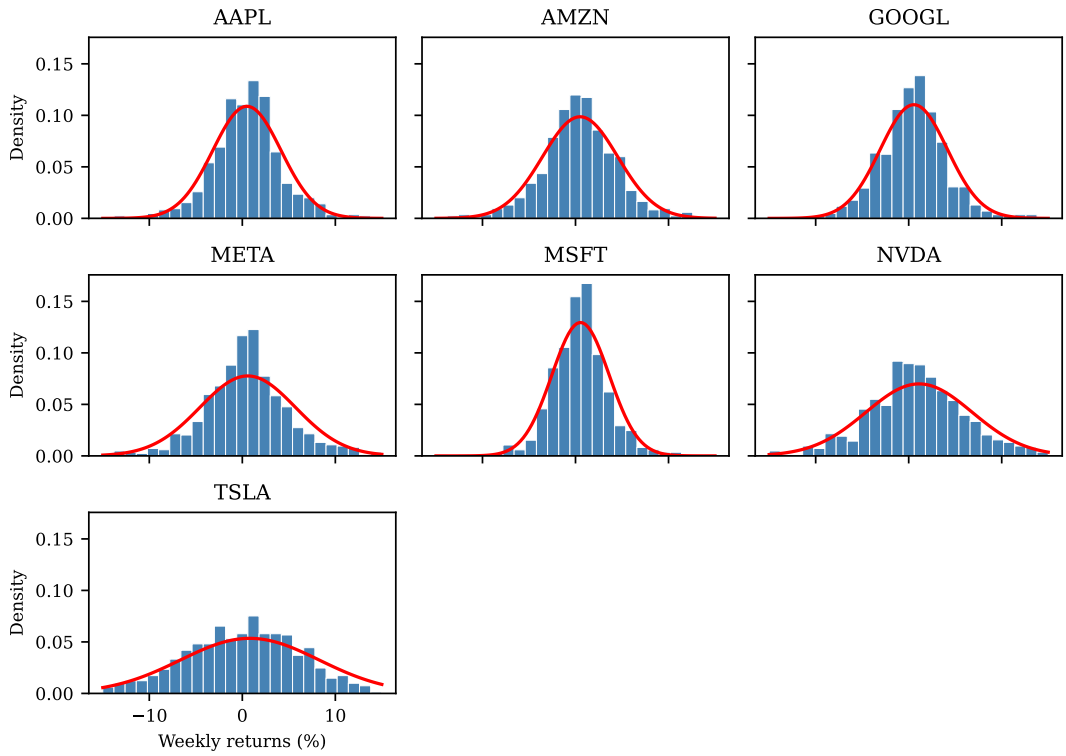
[37]:

Ticker	AAPL	AMZN	GOOGL	META	MSFT	NVDA	TSLA
mean	0.494079	0.494402	0.553937	0.578574	0.540312	1.113185	0.802365
std	3.666230	4.044092	3.614695	5.139174	3.080169	5.707891	7.461324

Once we have these statistics, we can add the normal PDF to each subplot. In this part, we use Matplotlib functions to create the figure from scratch, even though it is also possible to use pandas's `DataFrame.hist()` and add the PDF lines later.

[38]:
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm

# Get list of tickers from DataFrame columns
tickers = returns.columns.to_list()

# Fix number of columns, compute implied rows
ncol = 3
nrow = int(np.ceil(len(tickers) / ncol))

fig, axes = plt.subplots(nrow, ncol, figsize=(7, 5), sharex=True, sharey=True)

# x-values for plotting PDF
xvalues = np.linspace(xmin, xmax, 100)
```

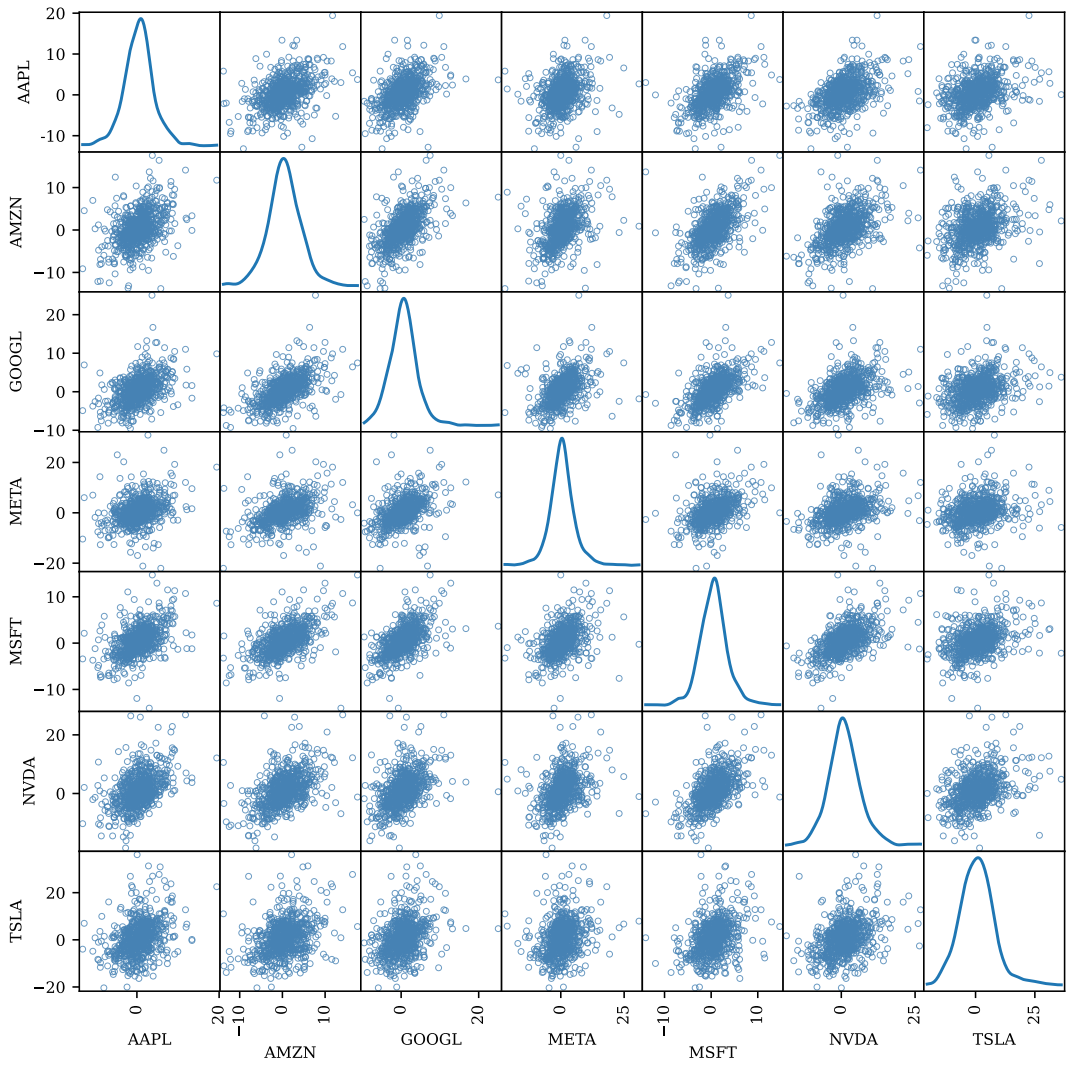
```
for i in range(nrow):
    for j in range(ncol):
        # Select current axes
        ax = axes[i, j]

        # Map axes index to ticker index
        k = i * ncol + j

        # Skip axes if there are no more tickers to plot
        if k >= len(tickers):
            ax.set_visible(False)
            continue

        # Ticker to plot in current axes
        ticker = tickers[k]

        # Retrieve moments for current ticker
        mean, std = moments[ticker]

        # Create histogram of returns
        ax.hist(
            returns[ticker],
            bins=25,
            density=True,
            lw=0.4,
            range=(xmin, xmax),
            color='steelblue',
            edgecolor='white',
        )

        # Superimpose PDF of normal distribution
        ax.plot(xvalues, norm.pdf(xvalues, mean, std), color='red', lw=1.5)

        # Add ticker name
        ax.set_title(ticker)

        # Add tick labels to the last row and first column
        if i == nrow - 1:
            ax.set_xlabel('Weekly returns (%)')
        if j == 0:
            ax.set_ylabel('Density')

fig.tight_layout()
```



Part 6 — Correlations

The simplest way to plot pairwise correlations is to use `scatter_matrix()` from the `pandas.plotting` module, as this does most of the work for us:

[39]:
```
from pandas.plotting import scatter_matrix

# Create figure with 7x7 scatter plots. Main diagonal shows kernel density
# for each ticker.
axes = scatter_matrix(
    returns,
    figsize=(9, 9),
    alpha=0.8,                  # Set transparency of markers
    color='none',               # Color of markers (no filling)
    edgecolors='steelblue',     # Color of marker edges
    lw=0.5,                     # Width of marker edges
    diagonal='kde',             # Add kernel density estimate to diagonal
)
```



From these scatter plots, you can see that for all seven stocks, weekly returns are positively correlated, although the strength of this correlation differs. To quantify these correlations, we compute the pairwise correlation matrix using `DataFrame.corr()`:

[40]:
```
# Compute pairwise correlations
corr = returns.corr()

# Tabulate correlations
corr
```

[40]:

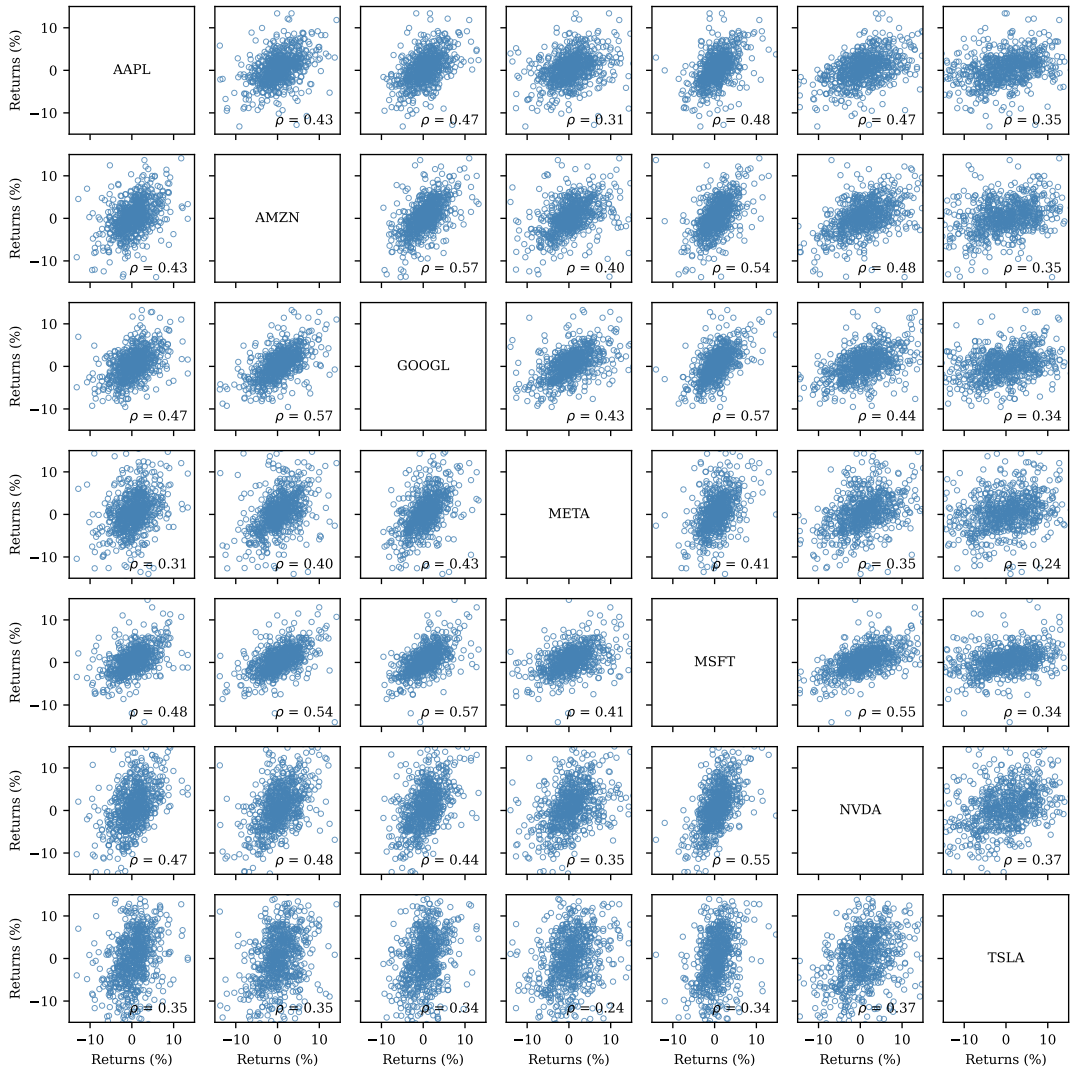
Ticker	AAPL	AMZN	GOOGL	META	MSFT	NVDA	TSLA
Ticker							
AAPL	1.000000	0.434487	0.469319	0.312579	0.484310	0.465806	0.348802
AMZN	0.434487	1.000000	0.574660	0.401116	0.541188	0.482258	0.346796
GOOGL	0.469319	0.574660	1.000000	0.428505	0.570993	0.440981	0.336179
META	0.312579	0.401116	0.428505	1.000000	0.410800	0.353203	0.239597
MSFT	0.484310	0.541188	0.570993	0.410800	1.000000	0.552622	0.343802
NVDA	0.465806	0.482258	0.440981	0.353203	0.552622	1.000000	0.373271
TSLA	0.348802	0.346796	0.336179	0.239597	0.343802	0.373271	1.000000

The following code recreates the 7-by-7 figure from above and adds this pairwise correlation as text to each subplot:

[41]:
```
# List of tickers present in DataFrame
tickers = returns.columns.to_list()
```

```
n = len(tickers)

fig, axes = plt.subplots(n, n, figsize=(9, 9), sharex=True, sharey=True)

# Set range for x- and y-axes
xmin, xmax = -15, 15

# Iterate over rows and columns
for i in range(n):
    for j in range(n):

        # Select current axes
        ax = axes[i, j]

        # Add tick labels to the last row and first column
        if i == n - 1:
            ax.set_xlabel('Returns (%)')
        if j == 0:
            ax.set_ylabel('Returns (%)')

        # For diagonal panels, print the ticker name instead of an
        # exactly diagonal scatter plot.
        if i == j:
            ax.text(
                0.5, 0.5, tickers[i], transform=ax.transAxes, va='center', ha='center'
            )
            continue

        # Get x- and y-values for this panel
        xvalues = returns.iloc[:, j]
        yvalues = returns.iloc[:, i]

        # Create scatter plot
        ax.scatter(
            xvalues,
            yvalues,
            s=10,
            alpha=0.8,
            lw=0.5,
            color='none',
            edgecolors='steelblue',
        )

        # Add text with pairwise correlation
        if i != j:
            ax.text(
                0.95,
                0.05,
                rf'$\rho$ = {corr.iloc[i, j]:.2f}',
                transform=ax.transAxes,
                va='bottom',
                ha='right',
            )

        # Set uniform x- and y-ticks for all axes
        ax.set_xlim((xmin, xmax))
        ax.set_ylim((xmin, xmax))
        ticks = -10, 0, 10
        ax.set_xticks(ticks)
        ax.set_yticks(ticks)
```

```
fig.tight_layout()
```



Exercise 4: Business cycle correlations

Use the macroeconomic data from the folder `../../data/FRED` to solve the following tasks:

1. There are seven decade-specific files named `FRED_monthly_19X0.csv` where `X` identifies the decade (`X` takes on the values 5, 6, 7, 8, 9, 0, 1). Write a loop that reads all seven files as DataFrames and stores them in a list.

   Hint: Recall that you can use `pd.read_csv(..., index_col='DATE', parse_dates=['DATE'])` to automatically parse strings stored in the `DATE` column as dates.

2. Use `pd.concat()` to concatenate these datasets into a single `DataFrame` and make sure that `DATE` is set as the index.

3. You realize that your data does not include GDP since this variable is only reported at quarterly frequency. Load the GDP data from the file `GDP.csv` and merge it with your monthly data using an *inner join*.

4. You want to compute how (percent) changes of the variables in your data correlate with percent changes in GDP.

1. Create a *new* `DataFrame` that contains the percent changes in CPI and GDP (using `pct_change()`), and the absolute changes for the remaining variables (using `diff()`).
2. Compute the correlation of the percent changes in GDP with the (percent) changes of all other variables (using `corr()`). What does the sign and magnitude of the correlation coefficient tell you?

Solution.

Part 1 — Load data files

[42]:
```
# Uncomment this to use files in the local data/ directory
DATA_PATH = '../../data/FRED'

# Uncomment this to load data directly from GitHub
# DATA_PATH = 'https://raw.githubusercontent.com/richardfoltyn/FIE463-V26/main/data/FRED'
```

There are many ways to load the seven files we need. One possibility is to loop over the decades 1950, 1960, ..., construct the decade-specific file name, and load the corresponding file:

[43]:
```
import numpy as np
import pandas as pd
import os.path

# Create years representing decades: 1950, 1960, ....
years = np.arange(1950, 2011, 10)

data = []
for year in years:
    # File name for current decade
    fn = f'FRED_monthly_{year}.csv'

    # Join data folder + filename to get path to CSV file
    path = os.path.join(DATA_PATH, fn)

    print(f'Loading file {path}')

    # Load decade data
    df = pd.read_csv(path, index_col='DATE', parse_dates=['DATE'])

    data.append(df)
```

```
Loading file ../../data/FRED/FRED_monthly_1950.csv
Loading file ../../data/FRED/FRED_monthly_1960.csv
Loading file ../../data/FRED/FRED_monthly_1970.csv
Loading file ../../data/FRED/FRED_monthly_1980.csv
Loading file ../../data/FRED/FRED_monthly_1990.csv
Loading file ../../data/FRED/FRED_monthly_2000.csv
Loading file ../../data/FRED/FRED_monthly_2010.csv
```

Instead of hard-coding the decades, we can also use `glob.glob()` to select files which match some pattern. The following code demonstrates how to load the files:

[44]:
```
import glob

# Pattern to match only the desired files in data/FRED. The wildcard *
# matches anything.
pattern = f'{DATA_PATH}/FRED_monthly_*.csv'

data = []
```

```
for file in glob.glob(pattern):
    print(f'Loading file {file}')
    d = pd.read_csv(file, index_col='DATE', parse_dates=['DATE'])
    data.append(d)

# Concatenate all DataFrames
df = pd.concat(data, axis=0)

# Sort index in case files have been loaded in unexpected order
df = df.sort_index()
```

```
Loading file ../../data/FRED/FRED_monthly_1950.csv
Loading file ../../data/FRED/FRED_monthly_1960.csv
Loading file ../../data/FRED/FRED_monthly_1970.csv
Loading file ../../data/FRED/FRED_monthly_1980.csv
Loading file ../../data/FRED/FRED_monthly_1990.csv
Loading file ../../data/FRED/FRED_monthly_2000.csv
Loading file ../../data/FRED/FRED_monthly_2010.csv
```

Part 2 — Concatenate data

[45]:
```
# Concatenate decade data along the row axis
df = pd.concat(data, axis=0)

# Print first 3 observations
df.head(3)
```

[45]:

DATE	CPI	UNRATE	FEDFUNDS	REALRATE	LFPART
1950-01-01	23.5	6.5	NaN	NaN	58.9
1950-02-01	23.6	6.4	NaN	NaN	58.9
1950-03-01	23.6	6.3	NaN	NaN	58.8

[46]:
```
# Print last 3 observations
df.tail(3)
```

[46]:

DATE	CPI	UNRATE	FEDFUNDS	REALRATE	LFPART
2019-10-01	257.2	3.6	1.8	-0.0	63.3
2019-11-01	257.9	3.6	1.6	-0.2	63.3
2019-12-01	258.6	3.6	1.6	-0.3	63.3

Part 3 — Load and merge GDP data

[47]:
```
# Path to GDP data
fn = os.path.join(DATA_PATH, 'GDP.csv')

# Load GDP data
gdp = pd.read_csv(fn, parse_dates=['DATE'], index_col='DATE')

# GDP data is at quarterly frequency
gdp.head(5)
```

[47]:

DATE	GDP
1947-01-01	2182.7
1947-04-01	2176.9
1947-07-01	2172.4
1947-10-01	2206.5

```
1948-01-01  2239.7
```

We merge the GDP using an *inner join*, which discards all months where GDP is not reported.

[48]:
```
# Merge the GDP data using an inner join
df = df.join(gdp, how='inner')

df.head(5)
```

[48]:

DATE	CPI	UNRATE	FEDFUNDS	REALRATE	LFPART	GDP
1950-01-01	23.5	6.5	NaN	NaN	58.9	2346.1
1950-04-01	23.6	5.8	NaN	NaN	59.2	2417.7
1950-07-01	24.1	5.0	NaN	NaN	59.1	2511.1
1950-10-01	24.5	4.2	NaN	NaN	59.4	2559.2
1951-01-01	25.4	3.7	NaN	NaN	59.1	2594.0

Part 4 — Compute changes & correlations

We can compute (percent) changes for multiple columns at once, so there is no need to even loop over variables:

[49]:
```
# Compute percent changes for CPI and GDP
df_changes = df[['CPI', 'GDP']].pct_change() * 100

# Other variables for which to compute absolute changes
variables = ['UNRATE', 'FEDFUNDS', 'REALRATE', 'LFPART']

# Compute absolute changes, add to DataFrame
df_changes[variables] = df[variables].diff()

df_changes.head(3)
```

[49]:

DATE	CPI	GDP	UNRATE	FEDFUNDS	REALRATE	LFPART
1950-01-01	NaN	NaN	NaN	NaN	NaN	NaN
1950-04-01	0.425532	3.051873	-0.7	NaN	NaN	0.3
1950-07-01	2.118644	3.863176	-0.8	NaN	NaN	-0.1

The `corr()` method returns the whole (symmetric) correlation matrix. We are only interested in the correlations with GDP changes, so we can select that particular row.

[50]:
```
# Compute correlation matrix, keep only GDP row
df_changes.corr().loc['GDP']
```

[50]:
```
CPI         -0.113091
GDP          1.000000
UNRATE      -0.564872
FEDFUNDS     0.206370
REALRATE     0.074500
LFPART       0.019639
Name: GDP, dtype: float64
```

As we can see, some (changes in) variables are more highly correlated with GDP changes than others. For example, the unemployment rate is highly negatively correlated with GDP growth, i.e., in good times (large positive GDP changes), the unemployment rate drops, as you would expect.

Exercise 5: Okun's Law

In this exercise, we investigate Okun's Law based on quarterly US data for each of the last seven decades.

Okun's Law relates unemployment to the output gap. One version (see Jones: Macroeconomics, 2019) is stated as follows:

$$\underbrace{u_t - \overline{u}_t}_{\text{cyclical unempl.}} = \alpha + \beta \underbrace{\left(\frac{Y_t - \overline{Y}_t}{\overline{Y}_t} \right)}_{\text{output gap}} \tag{3.1}$$

where u_t is the unemployment rate, $\overline{u}_t$ is the natural rate of unemployment, Y_t is output (GDP) and $\overline{Y}_t$ is potential output. We refer to $u_t - \overline{u}_t$ as "cyclical unemployment" and to the term in parentheses on the right-hand side as the "output gap." Okun's Law says that the coefficient β is negative, i.e., cyclical unemployment is higher when the output gap is low (negative) because the economy is in a recession.

Use the FRED data in the `../../data/FRED` folder and perform the following tasks:

1. Load the time series stored in `GDP.csv` (real GDP), `GDPPOT.csv` (real potential GDP), `UNRATE.csv` (unemployment rate) and `NROU.csv` (noncyclical rate of unemployment), where the last series corresponds to the natural rate of unemployment mentioned above.

   Combine these series into a single `DataFrame` so that each represents a column, and keep only observations from 1950-2019. The resulting data should be at quarterly frequency since GDP is only observed at these intervals.

   Hint: Use `pd.read_csv(..., index_col='DATE', parse_dates=['DATE'])` to automatically parse strings stored in the `DATE` column as dates and set it as the index.

2. Compute the output gap and cyclical unemployment rate as defined above, and add them as columns to the `DataFrame`.

   Plot these variables in a scatter plot with the output gap on the x-axis and the cyclical unemployment on the y-axis. Does Okun's Law hold over the sample period?

3. You wonder if the relationship has changed over the last decades. To answer this question, create a new column `Decade` which stores the decade of each observation, e.g., 1950, 1960, etc. Verify that each decade has 40 quarterly observations in your data.

   Hint: Since you have a date index, the calendar year can be retrieved from the attribute `df.index.year`.

4. Create a figure with 3-by-3 subplots showing the same scatter plot as above, but separately for each decade. Since we have data for only 7 decades, the last two subplots should remain empty.

5. **[Advanced]** Write a function `regress_okun()` which accepts a `DataFrame` containing a decade-specific sub-sample as the only argument, and estimates the coefficients α (the intercept) and β (the slope) of the above regression equation (3.1).

   This function should return a `Series` with two elements which store the intercept and slope.

   To run the regression by decade, group the data by `Decade` and call the `apply()` method, passing `regress_okun` that you wrote as the argument.

   Hint: Use NumPy's `lstsq()` to perform the regression. To regress the dependent variable `y` on regressors `X`, you need to call `lstsq(X, y)`. To include the intercept, you manually have to create `X` such that the first column contains only ones.

6. **[Advanced]** Plot your results: for each decade, create a scatter plot of the raw data and overlay it with the regression line you estimated.

Solution.

Part 1 — Load and process data

We load all four CSV files and concatenate them along the column dimension to get a single `DataFrame`.

[51]:
```
import pandas as pd

# Path to data directory
DATA_PATH = '../../data/FRED'

# Names of series to load
names = 'GDP', 'GDPPOT', 'UNRATE', 'NROU'
data = []

# Load the data frames
for s in names:
    df = pd.read_csv(f'{DATA_PATH}/{s}.csv', index_col=['DATE'], parse_dates=True)
    data.append(df)

# Concatenate the data frames along column axis
df = pd.concat(data, axis=1, join='inner')

# Restrict to desired time period
df = df.loc['1950':'2019']

# Print initial 10 rows of the merged data frame
df.head(10)
```

[51]:

DATE	GDP	GDPPOT	UNRATE	NROU
1950-01-01	2346.1	2380.5	6.5	5.3
1950-04-01	2417.7	2412.4	5.8	5.3
1950-07-01	2511.1	2443.5	5.0	5.3
1950-10-01	2559.2	2475.4	4.2	5.3
1951-01-01	2594.0	2507.4	3.7	5.3
1951-04-01	2638.9	2539.3	3.1	5.3
1951-07-01	2693.3	2572.9	3.1	5.3
1951-10-01	2699.2	2608.8	3.5	5.3
1952-01-01	2728.0	2646.6	3.2	5.3
1952-04-01	2733.8	2687.7	2.9	5.3

Part 2 — Compute and plot gaps

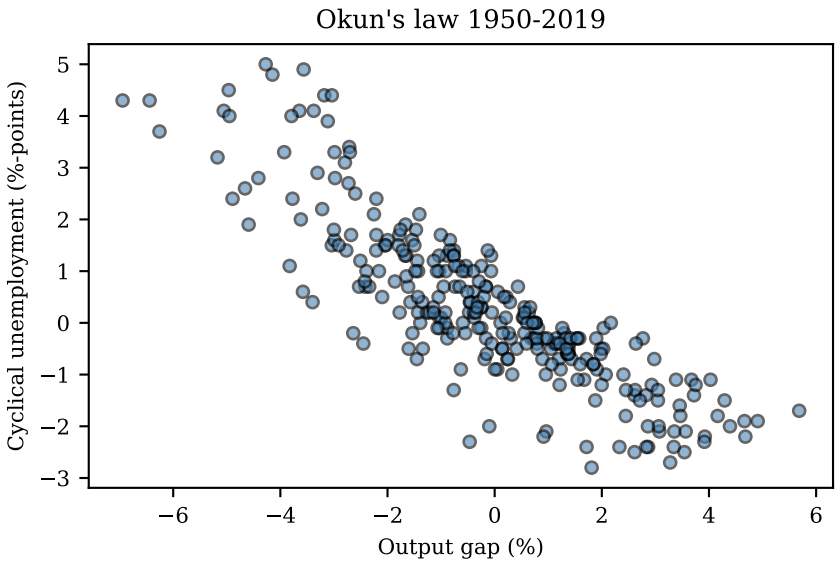
[52]:
```
# Output gap in percent
df['gdp_gap'] = (df['GDP'] - df['GDPPOT']) / df['GDPPOT'] * 100
# Cyclical unemployment in percentage points
df['u_gap'] = df['UNRATE'] - df['NROU']

# Print initial 3 rows
df.head(3)
```

[52]:

DATE	GDP	GDPPOT	UNRATE	NROU	gdp_gap	u_gap
1950-01-01	2346.1	2380.5	6.5	5.3	-1.445075	1.2
1950-04-01	2417.7	2412.4	5.8	5.3	0.219698	0.5
1950-07-01	2511.1	2443.5	5.0	5.3	2.766523	-0.3

We use the pandas plotting functions to create the scatter plot. Note that these functions are just wrappers around matplotlib and therefore support most of matplotlib's keyword arguments.

[53]:
```
# Use pandas scatter() method to create scatter plot
ax = df.plot.scatter(
    x='gdp_gap',                        # Column for x-axis
    y='u_gap',                          # Column for y-axis
    c='steelblue',
    edgecolors='black',                 # Color of marker edges
    s=20,                               # Marker size
    alpha=0.6,                          # Transparency
    figsize=(5, 3),
    xlabel='Output gap (%)',
    ylabel='Cyclical unemployment (%-points)',
    title='Okun\'s law 1950-2019',
)
```



As the graph shows, Okun's Law seems to hold during this period as high levels of cyclical unemployment are associated with large negative values of the output gap (i.e., the economy is in a recession).

Part 3 — Assign decade to each observation

The decade can be created from the calendar year by using floor division (//). The // operator performs division and rounds the result down to the nearest integer, so that `1951 // 10 = 195`, etc.

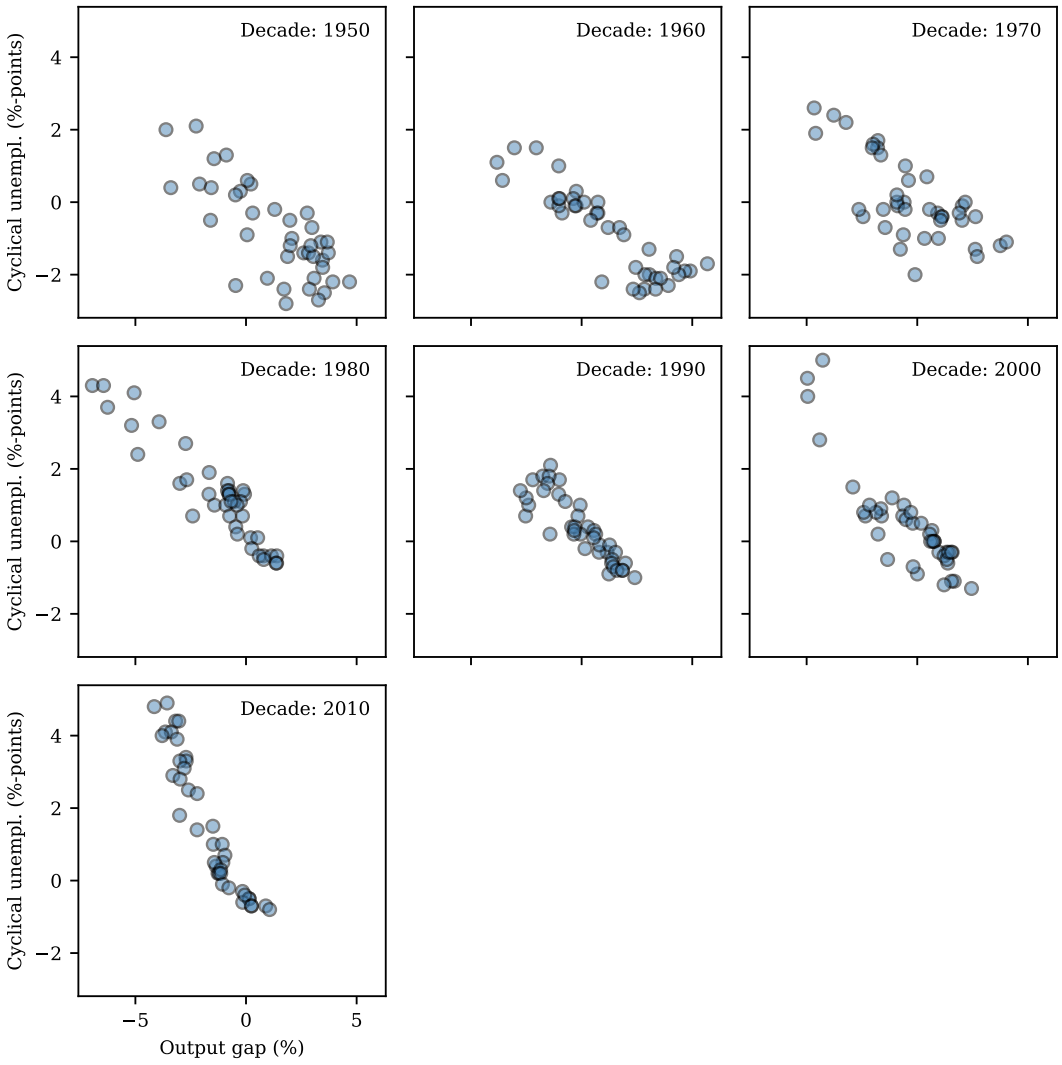
[54]:
```
# Create column to store the decade
df['Decade'] = df.index.year // 10 * 10

# Confirm that each decade has 40 quarterly observations
df['Decade'].value_counts()
```

[54]:
```
Decade
1950    40
1960    40
1970    40
1980    40
1990    40
2000    40
2010    40
Name: count, dtype: int64
```

Part 4 — Scatter plots by decade

We can now use the decade partitioning to create subplots by decade. The figure contains 3-by-3 subplots, but we have only 7 decades of data, so the last two subplots remain empty.

[55]:
```
import matplotlib.pyplot as plt
import numpy as np

decades = df['Decade'].unique()

ncol = 3
nrow = int(np.ceil(len(decades)/ncol))
fig, axes = plt.subplots(nrow, ncol, figsize=(6.5, 6.5), sharex=True, sharey=True)

for i in range(nrow):
    for j in range(ncol):
        # Select current axes
        ax = axes[i, j]

        # Map axes index to decade index
        k = i * ncol + j

        # Skip axes if there are no more decades to plot
        if k >= len(decades):
            ax.set_visible(False)
            continue

        # Decade to plot in current axes
        decade = decades[k]

        # Recover current decade
        df_decade = df.query(f'Decade == {decade}')

        # Scatter cyclical unemployment against output gap
        ax.scatter(df_decade['gdp_gap'], df_decade['u_gap'], alpha=0.5, s=30,
            c='steelblue', edgecolors='black')

        # Add decade label
        ax.text(0.95, 0.95, f'Decade: {decade}', transform=ax.transAxes, va='top',
 ↪ha='right')

        # Add tick labels to the last row and first column
        if i == nrow - 1:
            ax.set_xlabel('Output gap (%)')
        if j == 0:
            ax.set_ylabel('Cyclical unempl. (%-points)')


fig.tight_layout()
```



Part 5 — Run regressions by decade

We first write a function that accepts a subset of the `DataFrame` corresponding to each decade and runs the regression from (3.1)

[56]:
```
def regress_okun(x):
    """
    Run regression for Okun's Law.

    Parameters
    ----------
    x : DataFrame
        DataFrame with columns 'u_gap' and 'gdp_gap'.

    Returns
    -------
    Series
        Estimated coefficients.
    """

    # Extract dependent and regressor variables
    outcome = x['u_gap'].to_numpy()
    GDP_gap = x['gdp_gap'].to_numpy()
```

```
    # Regressor matrix including intercept
    regr = np.ones((len(GDP_gap), 2))
    # overwrite second column with output gap
    regr[:,1] = GDP_gap

    # Solve least-squares problem (pass rcond=None to avoid a warning)
    coefs, *rest = np.linalg.lstsq(regr, outcome, rcond=None)

    # Construct Series which will be returned to apply()
    columns = ['alpha', 'beta']
    df_out = pd.Series(coefs, index=columns)

    return df_out
```

[57]:
```
# Group by decade and apply regression to each group
coefs = df.groupby('Decade')[['gdp_gap', 'u_gap']].apply(regress_okun)

# Print estimated coefficients
coefs
```

[57]:

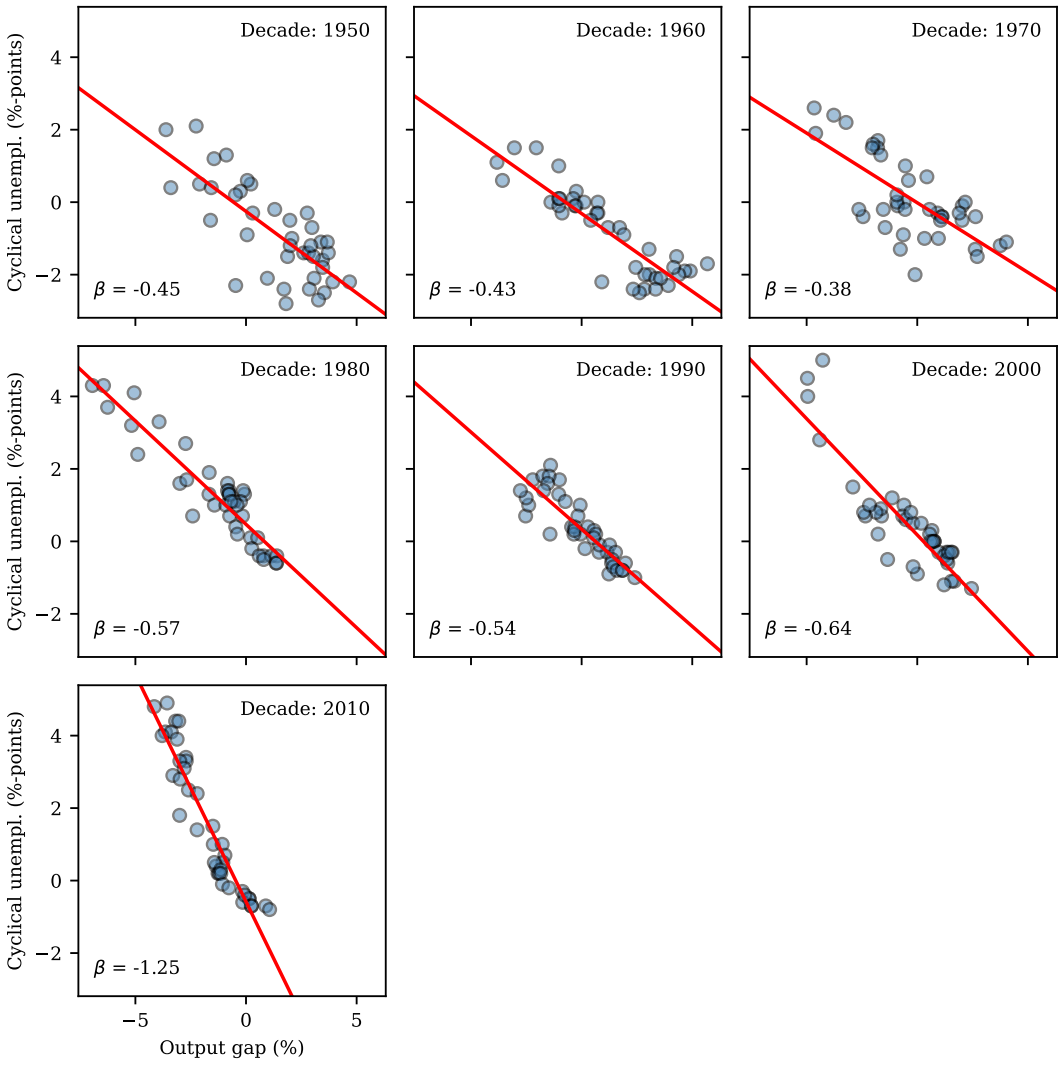
Decade	alpha	beta
1950	-0.254342	-0.450174
1960	-0.315124	-0.429339
1970	-0.016635	-0.384494
1980	0.470229	-0.570596
1990	0.333689	-0.536031
2000	0.177997	-0.641459
2010	-0.582794	-1.253611

Part 6 — Scatter plots with regression line

We now recreate the scatter plots from above but add the regression line to each subplot.

[58]:
```
fig, axes = plt.subplots(nrow, ncol, figsize=(6.5, 6.5), sharex=True, sharey=True)

for i in range(nrow):
    for j in range(ncol):
        ax = axes[i, j]

        # Decade index
        k = i * ncol + j

        # Skip axes if there are no more decades to plot
        if k >= len(decades):
            ax.set_visible(False)
            continue

        # Decade to plot in current axes
        decade = decades[k]

        # Recover current decade
        df_decade = df.query(f'Decade == {decade}')

        # Scatter cyclical unemployment against output gap
        ax.scatter(
            df_decade['gdp_gap'],
            df_decade['u_gap'],
            alpha=0.5,
            s=30,
```

```
            c='steelblue',
            edgecolors='black',
        )

        # Add regression line
        intercept, slope = coefs.loc[decade]
        ax.axline((0, intercept), slope=slope, color='red', linestyle='-', lw=1.5)

        # Add text with estimated slope of the regression line
        ax.text(
            0.05,
            0.05,
            f'$\\beta$ = {slope:.2f}',
            transform=ax.transAxes,
            va='bottom',
            ha='left',
        )

        # Add decade label
        ax.text(
            0.95,
            0.95,
            f'Decade: {decade}',
            transform=ax.transAxes,
            va='top',
            ha='right',
        )

        # Add tick labels to the last row and first column
        if i == nrow - 1:
            ax.set_xlabel('Output gap (%)')
        if j == 0:
            ax.set_ylabel('Cyclical unempl. (%-points)')


fig.tight_layout()
```


Decade: 1950
β = -0.45
Decade: 1960
β = -0.43
Decade: 1970
β = -0.38
Decade: 1980
β = -0.57
Decade: 1990
β = -0.54
Decade: 2000
β = -0.64
Decade: 2010
β = -1.25
Cyclical unempl. (%-points)
Output gap (%)